Novel Car Design

ES 3011 Final Project

Alex Fenwick, Felix Schroeder, Samuel Helayel

12/13/2024

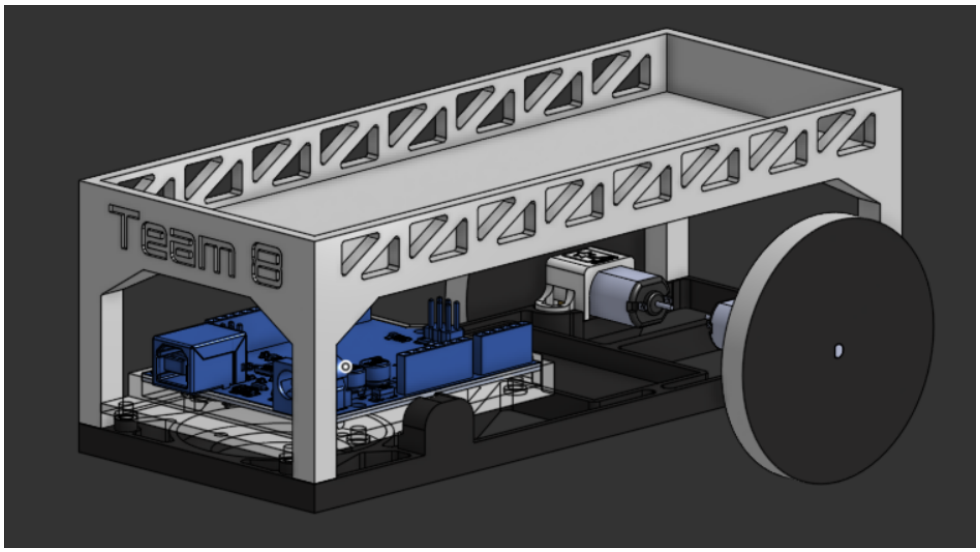
Introduction

For this project we designed our own car to fit the needs of this project. We had a few reasons for this redesign:

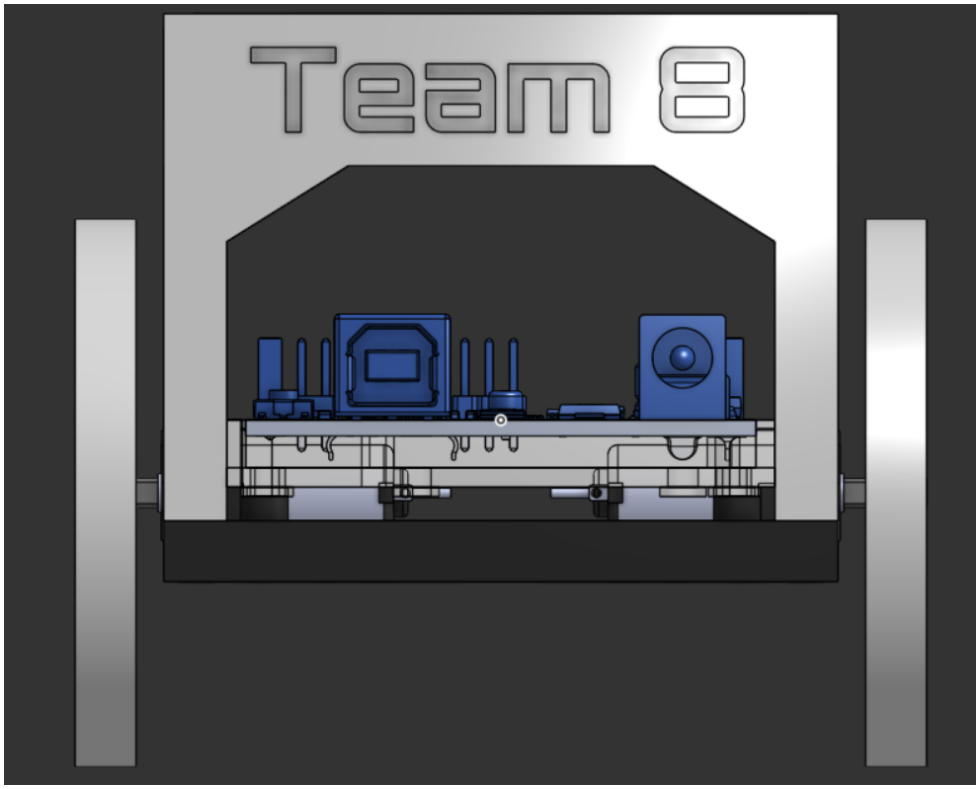
- Given car was not level, causing the blocks to be more prone to slipping and toppling
- Given car had a bunch of electronics up on tables, increasing the height of the center of gravity and making it more unstable
- Aesthetics
- Carrying capacity(until was later specified that blocks could only be stacked vertically)
- Easier Arduino mounting that make sure it remains fixed

Cad Pictures

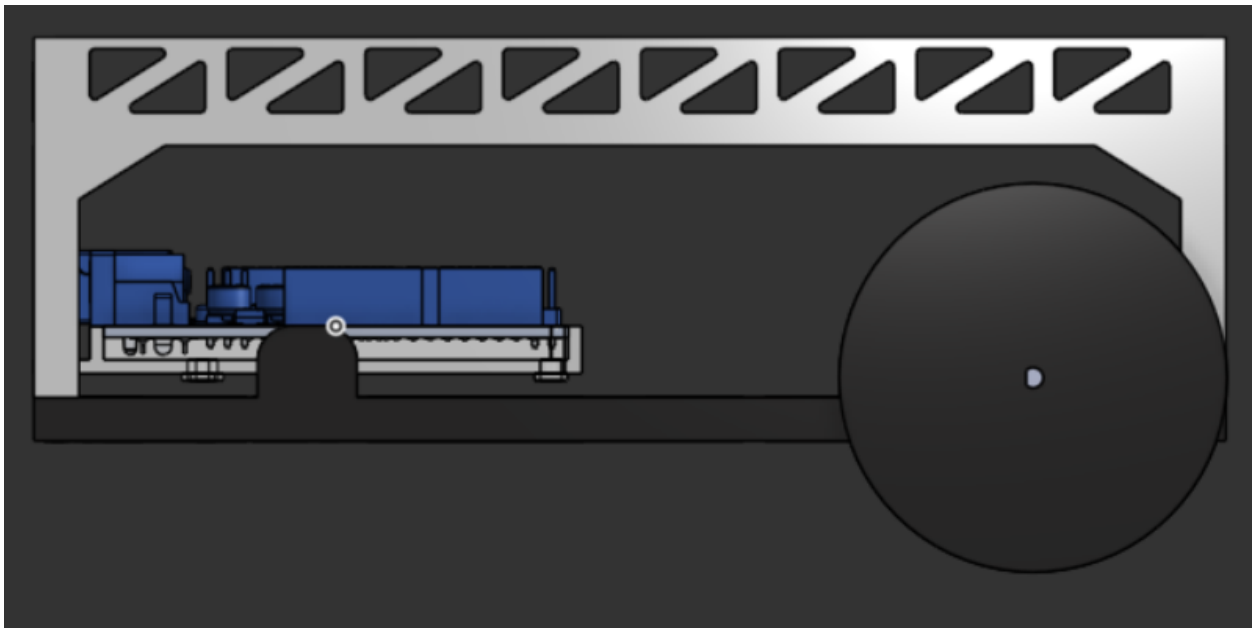
Isometric View:



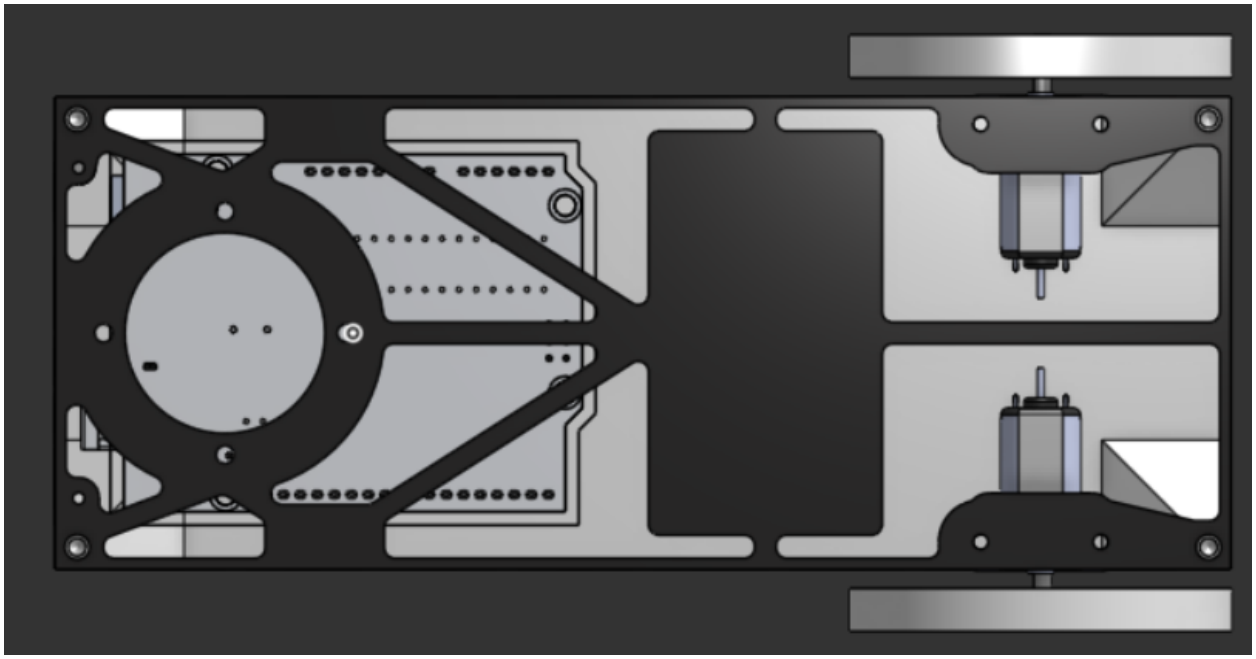
Front View:



Side View:

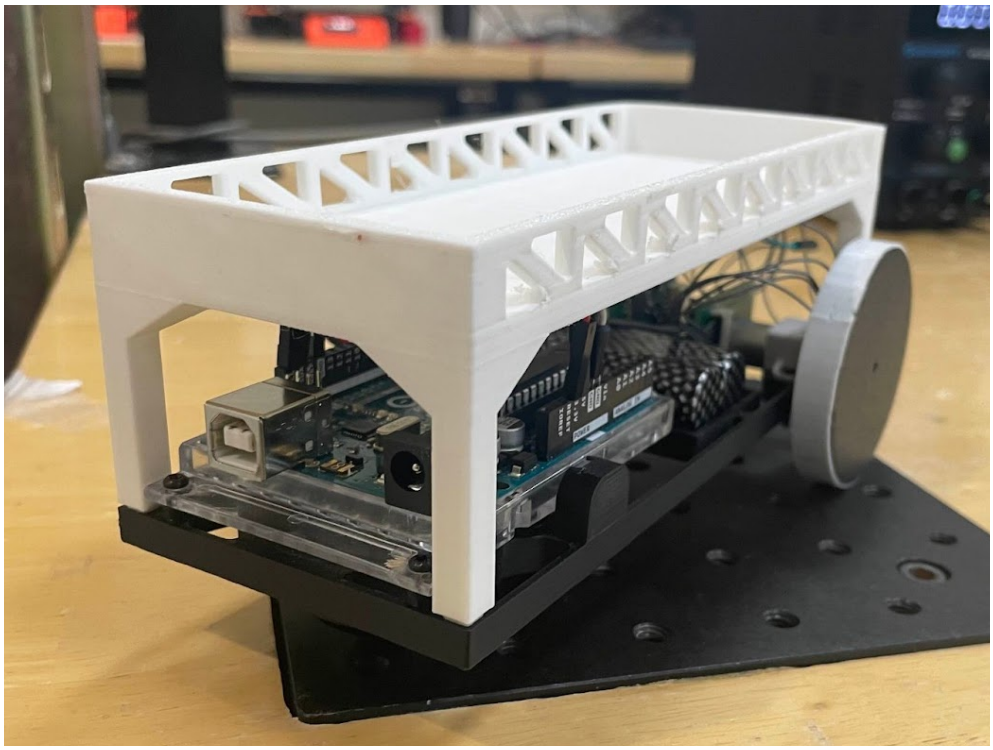


Bottom View:



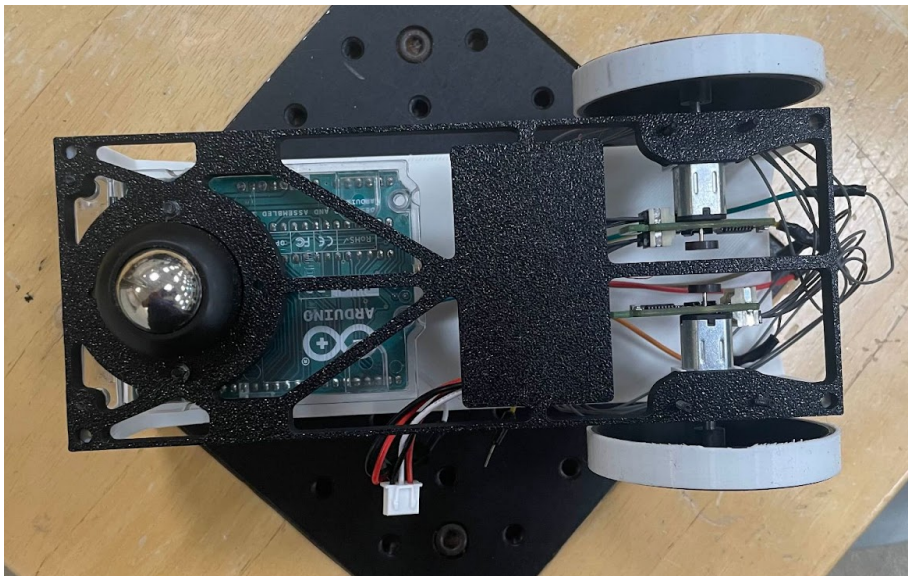
IRL Pictures

Isometric:



Side:

Bottom:



Video:

Test Run
Six Block Run
No Block Run

Programming

Here is the code that we used during our project

```
#include <Arduino.h>
#include <Wire.h>
#include <smartmotor.h>

// MOTOR PROPERTIES
#define GEAR_RATIO 150           // MOTOR GEAR RATIO
#define ENCODER_TICKS_PER_REV 12 // NO. OF HIGH PULSES PER ROTATION
const int32_t ENCODER_TICKS_PER_SHAFT_REV= ENCODER_TICKS_PER_REV * GEAR_RATIO;
#define DELAY_PERIOD 200

// INIT SMART MOTORS
SmartMotor motors[] = {0x08, 0x0B}; // INIT MOTOR W/ DEFAULT ADDRESS
// SmartMotor motors[] = {0x05,0x06,0x07}; // INIT MOTOR W/ DEFAULT ADDRESS
const int MOTOR_NUM = sizeof(motors)/sizeof(motors[0]);


volatile float target_pos_motor_0= -13333;
volatile float target_pos_motor_1= 13333;
volatile bool hasTurned = 0;
volatile bool goForward = 0;
float tol = 100;
float arc_length = 1260;

void setup() {
    // motors[0].reset();
    // motors[1].reset();
    Serial.begin(115200);

    Wire.begin(); // INIT DEVICE AS I2C CONTROLLER
    motors[0].set_address(0x08);
    motors[1].set_address(0x0B);

    delay(7000);
    // Sets PID Vars
    motors[0].tune_pos_pid(10, 0, 0);
```

```
    motors[1].tune_pos_pid(35, 6, 0.5);
    motors[0].tune_vel_pid(50, 10, 3);
    motors[1].tune_vel_pid(50, 10, 3);
}



void loop() {
    delay(DELAY_PERIOD);

    // Resets PID Loop
    motors[1].set_position(target_pos_motor_1);
    motors[0].set_position(target_pos_motor_0);

    // Gets
    float current_pos_0 = motors[0].get_position();
    float current_pos_1 = motors[1].get_position();

    Serial.println("-----------------------------------");

    if((abs(target_pos_motor_0 - current_pos_0) <= tol) &&
(abs(target_pos_motor_1 - current_pos_1) <= tol) && !hasTurned){
        Serial.println("GOD HELP ME!!");
        target_pos_motor_0 = target_pos_motor_0 + arc_length;
        target_pos_motor_1 = target_pos_motor_1 + arc_length;
        hasTurned = 1;
        goForward = 1;
    }



    if((abs(target_pos_motor_0 - current_pos_0) <= tol) &&
(abs(target_pos_motor_1 - current_pos_1) <= tol) && goForward){
        Serial.println("LORD HELP ME!!");
        target_pos_motor_0 = target_pos_motor_0 - 13333;
        target_pos_motor_1 = target_pos_motor_1 + 13333;
    }
```

```
    /*
    // for manual toggling, Should not run
    if (0 == 1) {
      if (abs(current_pos_0) - abs(current_pos_1) > 5) {
        motors[1].set_position(current_pos_1);
        motors[0].set_position(current_pos_1);
      } else if (abs(current_pos_0) - abs(current_pos_1) < 2) {
        motors[1].set_position(current_pos_0);
        motors[0].set_position(current_pos_0);
      } else {
        motors[1].set_position(target_pos_motor_1);
        motors[0].set_position(target_pos_motor_0);
      }
    }*/
}
```

Conclusion

Some of the challenges we ran into during this project were some mechanical/electrical ones, and we worked around changing wires to trying new motors in an attempt to find a solution. We found the issue was related to the motors forgetting their addresses randomly, so we fixed that and continued on with our project. We did some calculations to relate encoder counts to distance traveled, and found the target encoder distance needed to go a meter, and to turn. Once the logic and the code of the robot was figured out, we then began to test our robot and tweak the PID parameters to get as smooth of a robot as we could. We found some PID values that we were happy with (As shown in the code) but we still ran into issues where one of the motors was innately slower than the other, so despite our best efforts, the robot drifted slightly. Overall however, the experience allowed us to troubleshoot, find issues, debug, and work with PID parameters to get smooth motion.